

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»**

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

«ПРОЦЕДУРЫ, ФУНКЦИИ, ТРИГГЕРЫ В PostgreSQL»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Тихонов Роман Александрович

Факультет Прикладной информатики

Группа K3239

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

Преподаватель Говорова Марина Михайловна

Санкт-Петербург

2025/2026

Цель работы:

Овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание:

1. Создать 3 процедуры для индивидуальной БД «Курсы»:
 - получение расписания занятий для группы на заданный день недели;
 - запись слушателя на курс (в группу);
 - получение перечня свободных лекционных аудиторий на любой день недели (при отсутствии — вывод сообщения).
2. Создать 7 триггеров (вариант 2.1) для обеспечения целостности и автоматизации контроля данных.

Выполнение:

Наименование БД: Courses.

Вариант: 7.

Скрипт разработанных объектов (процедуры + триггеры):

<https://drive.google.com/file/d/1Dex9C4mRiHN56kfQwSs9tOIteFrQsrj/view?usp=sharing>

1. Реализация процедур по индивидуальному заданию, ч. 4:

Формулировка	процедуры	№	1:
lr3.pr_get_group_schedule_by_weekday(p_group_number, p_isodow, p_cur)			
— получает расписание занятий заданной группы на указанный день недели.			

SQL-команда теста процедуры № 1:

```
BEGIN;  
CALL lr3.pr_get_group_schedule_by_weekday('ФР-24', 3, 'c_sched');  
FETCH ALL FROM c_sched;  
COMMIT;
```

Результат выполнения теста процедуры № 1:

```

Courses=# BEGIN;
BEGIN
Courses=# CALL lr3.pr_get_group_schedule_by_weekday('ФР-24', 3, 'c_sched');
p_cur
-----
c_sched
(1 строка)

Courses=# FETCH ALL FROM c_sched;
group_number | lesson_date | lesson_time | discipline_name | lesson_type | room_number | room_type | teacher_fio
-----+-----+-----+-----+-----+-----+-----+-----
ФР-24       | 2024-04-10 | 10:00:00   | Веб-разработка | Лекция      | 301        | Лекционная | Сидоров Олег Дмитриевич
(1 строка)

Courses=# COMMIT;
COMMIT
Courses=#

```

Формулировка процедуры № 2: lr3.pr_enroll_trainee(...) — выполняет запись (зачисление) слушателя в группу:

- находит group_id по group_number;
- проверяет, что группа существует и не имеет статуса «Закрыта»;
- проверяет уникальность email (если задан);
- вставляет запись в lr3.trainee;
- вставляет паспорт в lr3.trainee_passport;
- возвращает созданный trainee_id через OUT-параметр o_trainee_id.

SQL-команда теста процедуры № 2:

```

CALL lr3.pr_enroll_trainee(
    'ФР-24'::varchar,
    'Иванов'::varchar,
    'Иван'::varchar,
    'Иванович'::varchar,
    '2000-01-01'::date,
    '89990000000'::varchar,
    'ivanov_test_1@mail.ru'::varchar,
    '1234'::varchar,
    '567890'::varchar,
    'ГУ МВД'::varchar,
    '2016-02-02'::date,
    NULL::int
);

SELECT trainee_id, last_name, first_name, email, group_id

```

```

FROM lr3.trainee
WHERE email='ivanov_test_1@mail.ru';

SELECT tp.*
FROM lr3.trainee_passport tp
JOIN lr3.trainee t ON t.trainee_id=tp.trainee_id
WHERE t.email='ivanov_test_1@mail.ru';

```

Результат выполнения теста процедуры № 2:

```

Courses=# CALL lr3.pr_enroll_trainee(
Courses(#  'ФП-24'::varchar,
Courses(#  'Иванов'::varchar,
Courses(#  'Иван'::varchar,
Courses(#  'Иванович'::varchar,
Courses(#  '2000-01-01'::date,
Courses(#  '89990000000'::varchar,
Courses(#  'ivanov_test_1@mail.ru'::varchar,
Courses(#  '1234'::varchar,
Courses(#  '567890'::varchar,
Courses(#  'ГУ МВД'::varchar,
Courses(#  '2016-02-02'::date,
Courses(#  NULL::int
Courses(# );
 o_trainee_id
-----
          4
(1 строка)

Courses=# SELECT trainee_id, last_name, first_name, email, group_id
Courses=# FROM lr3.trainee
Courses=# WHERE email='ivanov_test_1@mail.ru';
 trainee_id | last_name | first_name | email | group_id
-----+-----+-----+-----+-----
          4 | Иванов   | Иван      | ivanov_test_1@mail.ru | 1
(1 строка)

Courses=# SELECT tp.*
Courses=# FROM lr3.trainee_passport tp
Courses=# JOIN lr3.trainee t ON t.trainee_id=tp.trainee_id
Courses=# WHERE t.email='ivanov_test_1@mail.ru';
 trainee_passport_id | passport_series | passport_number | issued_by | issue_date | trainee_id
-----+-----+-----+-----+-----+-----
          3 | 1234          | 567890          | ГУ МВД   | 2016-02-02 | 4
(1 строка)

```

Формулировка процедуры № 3:

lr3.pr_get_free_lecture_rooms_by_weekday(p_isodow, p_time, p_cur) —
выводит перечень свободных лекционных аудиторий на ближайшую дату
(>= current_date) заданного дня недели.

- если p_time IS NULL — аудитории, свободные весь день (в этот день нет занятий в этой аудитории);

- если p_time задано — аудитории, свободные на указанное время.

SQL-команда теста процедуры № 3:

```
BEGIN;
CALL lr3.pr_get_free_lecture_rooms_by_weekday(1, '10:00', 'c_rooms');
FETCH ALL FROM c_rooms;
COMMIT;

BEGIN;
CALL lr3.pr_get_free_lecture_rooms_by_weekday(1, NULL, 'c_rooms2');
FETCH ALL FROM c_rooms2;
COMMIT;
```

Результат выполнения теста процедуры № 3:

```
Courses=# BEGIN;
BEGIN
Courses=*# CALL lr3.pr_get_free_lecture_rooms_by_weekday(1, '10:00', 'c_rooms');
  p_cur
-----
  c_rooms
(1 строка)

Courses=*# FETCH ALL FROM c_rooms;
 target_date | room_id | room_number | capacity
-----+-----+-----+-----
  2026-05-18 |      1 | 301         |      50
(1 строка)

Courses=*# COMMIT;
COMMIT
Courses=# BEGIN;
BEGIN
Courses=*# CALL lr3.pr_get_free_lecture_rooms_by_weekday(1, NULL, 'c_rooms2');
  p_cur
-----
  c_rooms2
(1 строка)

Courses=*# FETCH ALL FROM c_rooms2;
 target_date | room_id | room_number | capacity
-----+-----+-----+-----
  2026-05-18 |      1 | 301         |      50
(1 строка)
```

2. Реализация триггеров (вариант 2.1):

Формулировка триггера № 1: tr_lesson_no_room_conflict (BEFORE INSERT/UPDATE ON lr3.lesson) — запрещает назначать занятие в аудиторию, если в этой аудитории уже есть занятие на ту же дату и время.

SQL-команда теста триггера № 1:

```
BEGIN;

INSERT INTO lr3.lesson(lesson_date, lesson_time, group_id,
discipline_id, teacher_id, room_id, lesson_type_id)
VALUES ('2024-04-10', '10:00:00', 2, 1, 2, 1, 2);

ROLLBACK;
```

Результат выполнения теста триггера № 1:

```
Courses=# BEGIN;
BEGIN
Courses=#
Courses=# INSERT INTO lr3.lesson(lesson_date, lesson_time, group_id, discipline_id, teacher_id, room_id, lesson_type_id)
Courses=# VALUES ('2024-04-10', '10:00:00', 2, 1, 2, 1, 2);
ОШИБКА: Конфликт расписания: аудитория 1 занята на 2024-04-10 10:00:00
КОНТЕКСТ: функция PL/pgSQL fn_lesson_no_room_conflict(), строка 11, оператор RAISE
Courses=!#
Courses=!# ROLLBACK;
ROLLBACK
```

Формулировка триггера № 2: tr_lesson_no_teacher_conflict (BEFORE INSERT/UPDATE ON lr3.lesson) — запрещает назначать преподавателю два занятия на одну дату и время.

SQL-команда теста триггера № 2:

```
BEGIN;

INSERT INTO lr3.lesson(lesson_date, lesson_time, group_id,
discipline_id, teacher_id, room_id, lesson_type_id)
VALUES ('2024-04-10', '10:00:00', 2, 1, 2, 1, 2);

ROLLBACK;
```

Результат выполнения теста триггера № 2:

```
Courses=# BEGIN;
BEGIN
Courses=#
Courses=# INSERT INTO lr3.lesson(lesson_date, lesson_time, group_id, discipline_id, teacher_id, room_id, lesson_type_id)
Courses=# VALUES ('2024-04-10', '10:00:00', 2, 1, 1, 2, 2);
ОШИБКА: Конфликт расписания: преподаватель 1 занят на 2024-04-10 10:00:00
КОНТЕКСТ: функция PL/pgSQL fn_lesson_no_teacher_conflict(), строка 11, оператор RAISE
Courses=!#
Courses=!# ROLLBACK;
ROLLBACK
```

Формулировка триггера № 3: tr_lesson_no_group_conflict (BEFORE INSERT/UPDATE ON lr3.lesson) — запрещает назначать группе два занятия на одну дату и время.

SQL-команда теста триггера № 3:

```
BEGIN;

INSERT INTO lr3.lesson(lesson_date, lesson_time, group_id,
discipline_id, teacher_id, room_id, lesson_type_id)
VALUES ('2024-04-10', '10:00:00', 1, 2, 2, 2, 2);

ROLLBACK;
```

Результат выполнения теста триггера № 3:

```
Courses=# BEGIN;
BEGIN
Courses=#
Courses=# INSERT INTO lr3.lesson(lesson_date, lesson_time, group_id, discipline_id, teacher_id, room_id, lesson_type_id)
Courses=# VALUES ('2024-04-10', '10:00:00', 1, 2, 2, 2, 2);
ОШИБКА: Конфликт расписания: у группы 1 уже есть занятие на 2024-04-10 10:00:00
КОНТЕКСТ: функция PL/pgSQL fn_lesson_no_group_conflict(), строка 11, оператор RAISE
Courses=!#
Courses=!# ROLLBACK;
```

Формулировка триггера № 4: tr_lesson_teacher_qualification (BEFORE INSERT/UPDATE ON lr3.lesson) — запрещает назначать занятие, если преподаватель не закреплён за дисциплиной в таблице lr3.discipline_teacher.

SQL-команда теста триггера № 4:

```
BEGIN;

INSERT INTO lr3.lesson(lesson_date, lesson_time, group_id,
discipline_id, teacher_id, room_id, lesson_type_id)
VALUES ('2024-04-11', '12:00:00', 2, 2, 1, 1, 2);

ROLLBACK;
```

Результат выполнения теста триггера № 4:

```

Courses=# BEGIN;
BEGIN
Courses=#
Courses=# INSERT INTO lr3.lesson(lesson_date, lesson_time, group_id, discipline_id, teacher_id, room_id, lesson_type_id)
Courses=# VALUES ('2024-04-11', '12:00:00', 2, 2, 1, 1, 2);
ОШИБКА: Преподаватель 1 не закреплён за дисциплиной 2 (discipline_teacher)
КОНТЕКСТ: функция PL/pgSQL fn_lesson_teacher_qualification(), строка 9, оператор RAISE
Courses=!#
Courses=!# ROLLBACK;
ROLLBACK
Courses=#

```

Формулировка триггера № 5: `tr_lesson_in_group_period` (BEFORE INSERT/UPDATE ON `lr3.lesson`) — запрещает создавать занятие, если дата занятия выходит за период обучения группы (`start_date..end_date`).

SQL-команда теста триггера № 5:

```

BEGIN;

INSERT INTO lr3.lesson(lesson_date, lesson_time, group_id,
discipline_id, teacher_id, room_id, lesson_type_id)
VALUES ('2024-01-10', '12:00:00', 1, 1, 1, 1, 1);

ROLLBACK;

```

Результат выполнения теста триггера № 5:

```

Courses=# BEGIN;
BEGIN
Courses=#
Courses=# INSERT INTO lr3.lesson(lesson_date, lesson_time, group_id, discipline_id, teacher_id, room_id, lesson_type_id)
Courses=# VALUES ('2024-01-10', '12:00:00', 1, 1, 1, 1, 1);
ОШИБКА: Дата занятия 2024-01-10 раньше начала обучения группы 1 (start_date=2024-02-01)
КОНТЕКСТ: функция PL/pgSQL fn_lesson_in_group_period(), строка 12, оператор RAISE
Courses=!#
Courses=!# ROLLBACK;
ROLLBACK
Courses=#

```

Формулировка триггера № 6: `tr_attestation_check` (BEFORE INSERT/UPDATE ON `lr3.attestation`) — контролирует корректность аттестации:

- дата аттестации должна быть в периоде обучения группы слушателя;
- дисциплина должна входить в программу слушателя (проверка через `lr3.program_discipline`).

SQL-команда теста триггера № 6:

```

BEGIN;

```

```
INSERT INTO lr3.attestation(trainee_id, discipline_id,
attestation_date, result, attestation_format_id)
VALUES (1, 2, '2024-05-01', 'PASS', 2);
```

```
ROLLBACK;
```

Результат выполнения теста триггера № 6:

```
Courses=# BEGIN;
BEGIN
Courses=#
Courses=# INSERT INTO lr3.attestation(trainee_id, discipline_id, attestation_date, result, attestation_format_id)
Courses=# VALUES (1, 2, '2024-05-01', 'PASS', 2);
ОШИБКА: Дисциплина 2 не входит в программу 1 (trainee_id=1)
КОНТЕКСТ: функция PL/pgSQL fn_attestation_check(), строка 31, оператор RAISE
Courses=!#
Courses=!# ROLLBACK;
```

Формулировка триггера № 7: tr_trainee_log (AFTER INSERT/UPDATE/DELETE ON lr3.trainee) — выполняет логирование операций над таблицей слушателей: при INSERT/UPDATE/DELETE добавляет текстовое сообщение в lr3.logs.

SQL-команда теста триггера № 7:

```
INSERT INTO lr3.trainee(last_name, first_name, middle_name,
birth_date, group_id, phones, email, trainee_status_id)
VALUES ('Тестов', 'Тест', NULL, '2002-02-02', 1, '800000000000',
'trainee_log_test@mail.ru', 1);
```

```
UPDATE lr3.trainee
SET phones = '81111111111'
WHERE email = 'trainee_log_test@mail.ru';
```

```
DELETE FROM lr3.trainee
WHERE email = 'trainee_log_test@mail.ru';
```

```
SELECT * FROM lr3.logs ORDER BY log_id DESC LIMIT 20;
```

Результат выполнения теста триггера № 7:

```

Courses=# INSERT INTO lr3.trainee(last_name, first_name, middle_name, birth_date, group_id, phones, email, trainee_status_id)
Courses=# VALUES ('Тестов', 'Тест', NULL, '2002-02-02', 1, '80000000000', 'trainee_log_test@mail.ru', 1);
INSERT 0 1
Courses=#
Courses=# UPDATE lr3.trainee
Courses=# SET phones = '81111111111'
Courses=# WHERE email = 'trainee_log_test@mail.ru';
UPDATE 1
Courses=#
Courses=# DELETE FROM lr3.trainee
Courses=# WHERE email = 'trainee_log_test@mail.ru';
DELETE 1
Courses=#
Courses=# SELECT * FROM lr3.logs ORDER BY log_id DESC LIMIT 20;
log_id | text | added
-----+-----+-----
4 | DELETE trainee: id=5, fio=Тестов Тест , group_id=1 | 2026-05-12 23:52:42.643901
3 | UPDATE trainee: id=5, fio=Тестов Тест , group_id: 1 -> 1 | 2026-05-12 23:52:42.636387
2 | INSERT trainee: id=5, fio=Тестов Тест , group_id=1 | 2026-05-12 23:52:42.631112
1 | INSERT trainee: id=4, fio=Иванов Иван Иванович, group_id=1 | 2026-05-12 23:42:38.250664
(4 строки)

```

Выводы:

В ходе выполнения лабораторной работы №5 были отработаны практические навыки создания и использования серверной логики в PostgreSQL и её проверки в консоли psql. На основе индивидуальной базы данных «Курсы» были разработаны и протестированы хранимые процедуры, обеспечивающие получение расписания занятий для группы на заданный день недели, выполнение записи слушателя в учебную группу с добавлением паспортных данных, а также поиск свободных лекционных аудиторий на выбранный день недели (с учётом времени занятия либо свободных аудиторий на весь день).

Также были реализованы триггеры, предназначенные для автоматического контроля корректности данных и соблюдения бизнес-правил предметной области. Триггеры предотвращают конфликты расписания по аудитории, преподавателю и учебной группе, проверяют соответствие преподавателя дисциплине, контролируют попадание занятий и аттестации в допустимый период обучения группы, а также выполняют автоматическое логирование изменений в таблице слушателей. Тестирование показало, что перенос проверок на уровень БД повышает надёжность системы и исключает внесение некорректных данных независимо от используемого клиентского приложения.

Цель работы достигнута: освоены разработка процедур и триггеров в PostgreSQL и методика их тестирования в psql.